

Unit VI: Web Services and APIs (5 hours)

- Introduction to Web Services
- Brief about service-oriented architecture
- SOAP
- RESTful web services
- RESTful APIs vs. SOAP
- JSON and XML data formats
- Building and Consuming APIs
 - Creating simple RESTful APIs using Flask

6.1 Introduction to Web Services

Web services is a collection of technologies, including XML, Simple Object Access Protocol (SOAP), Web Services Description Language (WSDL) and Universal Description, Discover and Integration (UDDI), which allow you to build programming solutions for specific messaging and application integration problems

Web Services are standardized software components or systems that allow different applications to communicate and exchange or share data over a network, typically on the Internet using open protocols (mainly HTTP/HTTPS) and structured formats like XML (or JSON). They act as a bridge between different software systems, regardless of the platform or programming language used. Web services are a fundamental part of **distributed computing** and **cloud-based architectures**, enabling **interoperability**, **modularity**, and **reusability** in modern software development.

Web Services have two types of uses

1. Reusable application-components

There are things applications needs very often. So why make these over and over again? Web services can offer application-components like: currency conversion, weather reports, or even language translation as services.

2. Connect existing software

Web services can help to solve the interoperability problem by giving different applications a way to link their data. With Web services you can exchange data between different applications and different platforms.

Key Points:

- Web services enable **application-to-application interaction** over the internet.
- They use **open standards** like XML, SOAP, WSDL, and HTTP.
- Commonly used for **exposing business logic** to remote clients.
- Platform- and language-independent.
- Examples include services for **weather data, payment gateways, currency converters, and product catalogs**.

Core Components:

1. **SOAP (Simple Object Access Protocol)**: Protocol for exchanging structured XML messages over HTTP.
2. **UDDI (Universal Description, Discovery, and Integration)**: A directory for publishing and discovering web services.
3. **WSDL (Web Services Description Language)**: XML-based file that describes the functionality and access methods of a web service.

How Web Services Work:

- Clients make **Remote Procedure Calls (RPCs)** via XML messages.
- Data is exchanged using **XML over HTTP**.
- Web services can be written in one language and accessed by another due to standardized communication.

Key Features:

- **XML-Based**: Platform- and language-independent.
- **Loosely Coupled**: Systems can evolve independently.
- **Synchronous or Asynchronous**: Clients can wait or continue after sending a request.
- **Coarse-Grained**: Encapsulates complex business logic.
- **Supports RPC and Document Exchange**: Suitable for both simple and complex data.

Advantages:

- **Internet Accessibility:** Functions available globally via HTTP.
- **Interoperability:** Works across different platforms (e.g., .NET, Java or any other).
- **Low Communication Cost:** Uses existing web infrastructure.
- **Standard Protocols:** Universally recognized protocols ensure consistency.
- **Reusability:** One service can be reused by multiple clients.

6.2 Service Oriented Architecture (SOA)

Service-Oriented Architecture (SOA) is a design paradigm in which software components (called services) are exposed over a network and communicate through standard protocols. Each service is a self-contained unit of functionality that can be discovered and used by other services or applications. SOA enables you to design software systems that provide services to other applications through published and discoverable interfaces, and where the services can be invoked over a network. When you implement an SOA using Web services technologies, you create a new way of building applications within a more powerful, flexible programming model. SOA helps build distributed systems where services are reusable, loosely coupled, and interoperable. You can reduce your development and ownership costs-and your implementation risk.

Components of SOA

- They offer business services across the platforms
- They provide location independence
- Services need not be at a particular system or particular network
- Completely loosely coupled approach
- Authentication and authorization support at every level
- The search and connectivity to other services is dynamic

6.3 Simple object Access Protocol (SOAP)

SOAP is a protocol used for exchanging structured information in the implementation of web services. It allows programs running on different operating systems and written in different programming languages to communicate over the Internet.

SOAP is the communications protocol for XML Web services. SOAP is described as a communications protocol. SOAP is a specification that defines the XML format for messages.

Simple Object Access protocol is a protocol specification for exchanging structured information in the implementation of web services in computer networks. It uses XML Information Set for its message format, and relies on other application layer protocols, most notably Hypertext Transfer Protocol (HTTP) or Simple Mail Transfer Protocol (SMTP), for message negotiation and transmission. SOAP is XML-based and typically relies on HTTP or SMTP for message transport.

SOAP has three major characteristics:

1. Extensibility (security and WS-routing are among the extensions under development)
2. Neutrality (SOAP can operate over any transport protocol such as HTTP, SMTP, TCP, UDP, or JMS)
3. Independence (SOAP allows for any programming model)

The SOAP architecture consists of several layers of specifications for:

- Message format
- Message Exchange Patterns (MEP)
- Underlying transport protocol bindings
- Message processing models
- Protocol extensibility

SOAP Elements

1. The SOAP Envelope Element

The required SOAP Envelope element is the root element of a SOAP message. This element defines the XML document as a SOAP message.

Example

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
...
  Message information goes here
```

```
...  
</soap:Envelope>
```

2. The SOAP Header Element

The optional SOAP Header element contains application-specific information (like authentication, payment, etc) about the SOAP message. If the Header element is present, it must be the first child element of the Envelope element.

Note: All immediate child elements of the Header element must be namespace-qualified.

```
<?xml version="1.0"?>  
<soap:Envelope  
  xmlns:soap="http://www.w3.org/2001/12/soap-envelope"  
  soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">  
  
  <soap:Header>  
    <m:Trans xmlns:m="http://www.w3schools.com/transaction/"  
      soap:mustUnderstand="1">234  
    </m:Trans>  
  </soap:Header>  
  
  ...  
  ...  
</soap:Envelope>
```

3. The SOAP Body Element:

The required SOAP Body element contains the actual SOAP message intended for the ultimate endpoint of the message. Immediate child elements of the SOAP Body element may be namespace-qualified.

Example

```
<?xml version="1.0"?>  
<soap:Envelope  
  xmlns:soap="http://www.w3.org/2001/12/soap-envelope"  
  soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
```

```
<soap:Body>
  <m:GetPrice xmlns:m="http://www.abc.com/prices">
    <m:Item>Apples</m:Item>
  </m:GetPrice>
</soap:Body>

</soap:Envelope>
```

4. The SOAP Fault Element:

The optional SOAP Fault element is used to indicate error messages. If a Fault element is present, it must appear as a child element of the Body element. A Fault element can only appear once in a SOAP message. The SOAP Fault element has the following sub elements:

Sub Element	Description
<faultcode>	A code for identifying the fault
<faultstring>	A human readable explanation of the fault
<faultactor>	Information about who caused the fault to happen
<detail>	Holds application specific error information related to the Body element

5. SOAP Fault Codes:

The fault code values defined below must be used in the fault code element when describing faults:

Error	Description
VersionMismatch	Found an invalid namespace for the SOAP Envelope element
MustUnderstand	An immediate child element of the Header element, with the mustUnderstand attribute set to "1", was not understood
Client	The message was incorrectly formed or contained incorrect information
Server	There was a problem with the server so the message could not proceed

6.4 REST ful Web services:

The term *Web services* is typically associated with operation or action-based services using SOAP and WSDL. The term *REST Web services* generally refer to a resource-based Web services architecture that uses HTTP and XML. Each of these architectural Web service styles has its place but until recently, the WSDL standard didn't equally support both styles. The WSDL binding was inadequate to describe communications with HTTP and XML, so there was no way to formally describe REST Web services with WSDL. The publication of WSDL which was designed with REST Web services and World Wide Web Consortium (W3C) recommendation means there is now a language to describe REST Web services.

REST is an architectural style that treats the Web as a resource-centric application. Practically, this means each URL in a RESTful application represents a resource. The URLs are also easy to understand and remember.

- RESTful Web services make use of hyperlinks in XML.
- RESTful applications access resources following the create, read, update, and delete (CRUD) style using the full range of HTTP verbs (POST, GET, PUT, and DELETE).
- RESTful applications should be stateless.
- REST application no session state is stored on the server.

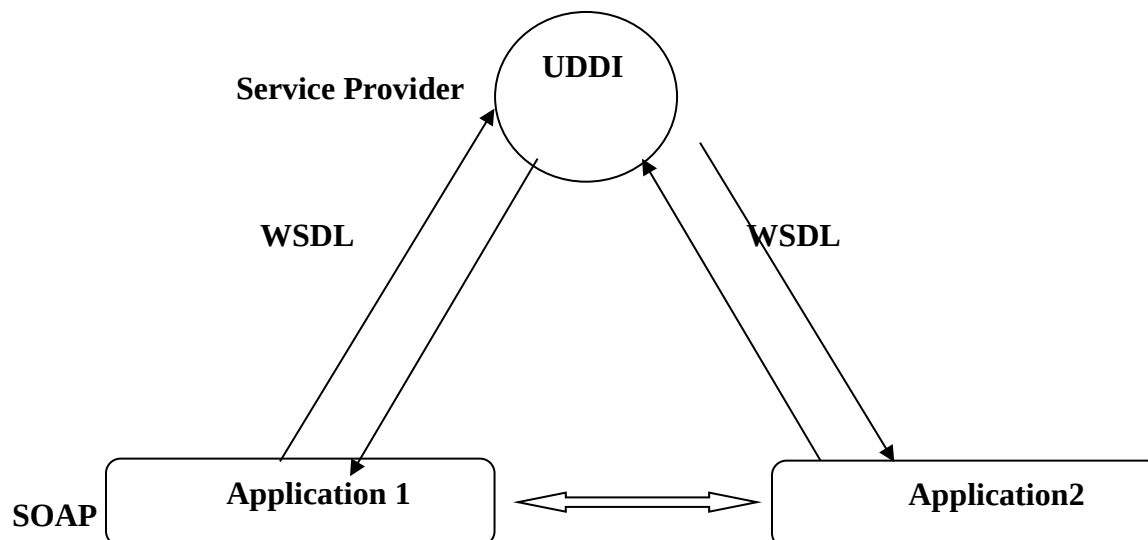


Fig: Configuration of Web-Services, SOA and REST ful

6.5 RESTful APIs vs. SOAP

REST (Representational State Transfer) and SOAP (Simple Object Access Protocol) are protocols used for exchanging structured information between systems over a network. However, they differ significantly in design, implementation, and use cases.

Feature	RESTful API	SOAP
Definition	REST (Representational State Transfer) is an architectural style using standard HTTP methods.	SOAP is a protocol using XML for message format and relies on various application layer protocols like HTTP, SMTP.
Protocol	Not a protocol; it's a style that typically uses HTTP.	Strict protocol with defined rules and standards.
Message Format	JSON, XML, HTML, or plain text (commonly JSON).	Strictly XML.
Ease of Use	Simpler to use, especially for web-based services.	More complex due to strict standards.
Performance	Lightweight and faster; ideal for mobile and web applications.	Heavier; slower because of XML parsing and envelope processing.
Security	Relies on HTTPS for transport-level security; OAuth and JWT often used.	Built-in WS-Security (supports encryption, signing, etc.). More suitable for enterprise-level apps.
Flexibility	More flexible in terms of data formats and usage.	Rigid due to tight specifications.
Statefulness	Typically, stateless (each call is independent).	Can be stateless or stateful depending on use.
Error Handling	Uses HTTP status codes (e.g., 404, 500).	Detailed error information in the XML response body.
Standards Support	No official standard; it's a convention.	Official W3C standard.

Best For	Lightweight web services, mobile apps, public APIs.	Enterprise-level services requiring high security, reliability, and ACID compliance.
----------	---	--

Conclusion:

- **Use REST** when you want simplicity, speed, and flexibility ideal for modern web/mobile development.
- **Use SOAP** when you need **strong security, formal contracts, transaction compliance,** and **robustness** typical in **banking, telecom, or enterprise services.**

6.6 JSON and XML data formats

JSON (JavaScript Object Notation) and XML (eXtensible Markup Language) are widely used for data interchange between systems, especially in web services and APIs.

Feature	JSON (JavaScript Object Notation)	XML (eXtensible Markup Language)
Format Type	Lightweight data-interchange format	Markup language
Readability	Easy to read and write (compact and clean)	More verbose; uses opening and closing tags
Data Representation	Objects (key-value pairs), arrays	Hierarchical structure with nested tags
Syntax	JavaScript-like syntax; uses { }, []	Uses <tags> for elements
Data Types	Supports strings, numbers, booleans, arrays, objects	All values are treated as text (must be parsed manually into data types)
Schema Support	JSON Schema (optional)	XML Schema Definition (XSD) is well-established
Comments	Not supported in standard JSON	Supported
Parsing	Faster and easier in most languages	Slower due to document structure and verbosity

Usage	Common in REST APIs, web apps, mobile apps	Common in SOAP, document storage, configuration
Size	Smaller file size due to compact syntax	Larger file size due to verbose tags
Support	Native in JavaScript; widely supported across platforms	Supported by most programming languages and platforms

Example

JSON

```
{
  "student": {
    "name": "Sunil",
    "age": 21,
    "courses": [
      "Math",
      "Science"
    ]
  }
}
```

XML

```
<student>
  <name>Sunil</name>
  <age>21</age>
  <courses>
    <course>Math</course>
    <course>Science</course>
  </courses>
</student>
```

JSON Uses:

- You want lightweight, readable, and fast data interchange.
- You are working with web technologies or REST APIs.

XML Uses:

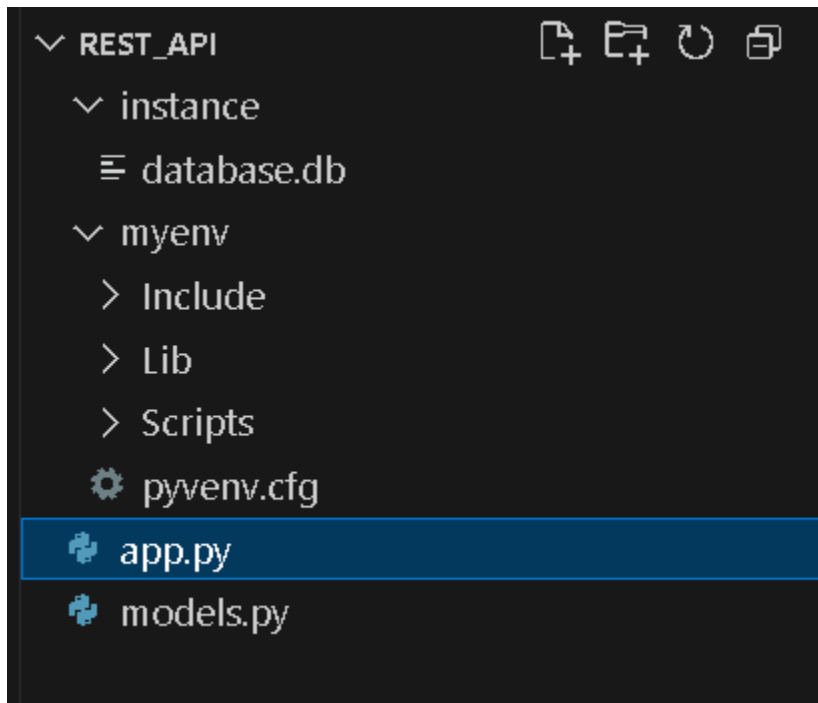
- You need strict validation, complex document structures, or metadata.
- You're working with legacy systems or SOAP APIs.

6.7 Building and Consuming APIs

6.7.1 Creating simple RESTful APIs using Flask

A RESTful API (Representational State Transfer) uses HTTP methods like GET, POST, PUT, and DELETE to perform operations on resources (data entities like users, products, etc.).

Create folder structure as:



Create and Activate virtual environment and install the packages
pip install flask flask_sqlalchemy

Install postman client for API test (or any other software)

<https://www.postman.com/downloads/>

models.py

```
from flask_sqlalchemy import SQLAlchemy  
  
db = SQLAlchemy()
```

```

class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    age = db.Column(db.Integer, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

    def to_dict(self):
        return {"id": self.id, "name": self.name, "age": self.age, "email": self.email}

```

app.py

```

from flask import Flask, request, jsonify
from models import db, Student

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db.init_app(app)

# Create a student
@app.route('/students', methods=['POST'])
def create_student():
    data = request.get_json()
    new_student = Student(name=data['name'], age=data['age'],
email=data['email'])
    db.session.add(new_student)
    db.session.commit()
    return jsonify(new_student.to_dict()), 201

# Get all students
@app.route('/students', methods=['GET'])
def get_students():
    students = Student.query.all()
    return jsonify([s.to_dict() for s in students])

# Get a single student
@app.route('/students/<int:id>', methods=['GET'])
def get_student(id):
    student = Student.query.get_or_404(id)
    return jsonify(student.to_dict())

# Update a student
@app.route('/students/<int:id>', methods=['PUT'])

```

```

def update_student(id):
    student = Student.query.get_or_404(id)
    data = request.get_json()
    student.name = data.get('name', student.name)
    student.age = data.get('age', student.age)
    student.email = data.get('email', student.email)
    db.session.commit()
    return jsonify(student.to_dict())

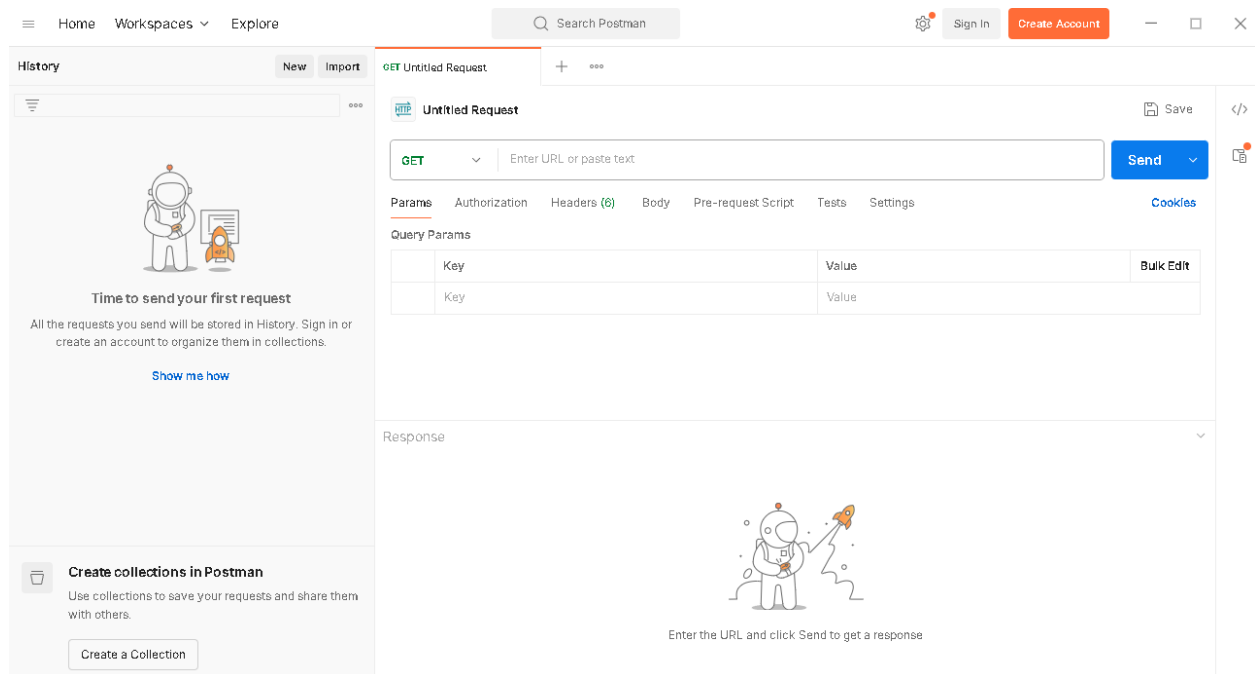
# Delete a student
@app.route('/students/<int:id>', methods=['DELETE'])
def delete_student(id):
    student = Student.query.get_or_404(id)
    db.session.delete(student)
    db.session.commit()
    return jsonify({'message': 'Student deleted successfully'})

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(debug=True)

```

Run the program

Open postman client and it will show as below:



URL End Points

GET: /students => list all students

GET: /students/1 => display individual records of student with id 1

POST: /students => Save student record to database

PUT: /students/1 => Update student record

DELETE: /student/1 => Delete student record with id 1

http://127.0.0.1:5000/students:GET

```
[
  {
    "age": 33,
    "email": "sunil@gmail.com",
    "id": 1,
    "name": "Sunil Bahadur Bist"
  },
  {
    "age": 22,
    "email": "sunil@nast.edu.np",
    "id": 2,
    "name": "Sunil Bist"
  }
]
```

http://127.0.0.1:5000/students/1:GET

```
{  
  "age": 33,  
  "email": "sunil@gmail.com",  
  "id": 1,  
  "name": "Sunil Bahadur Bist"  
}
```

http://127.0.0.1:5000/students:POST

```
{  
  "name": "Sunil Bist",  
  "age": 22,  
  "email": "sunil@nast.edu.np"  
}
```

http://127.0.0.1:5000/students/1:PUT

```
{  
  "age": 33,  
  "email": "sunil@gmail.com",  
  "id": 1,  
  "name": "Sunil Bahadur Bist"  
}
```

http://127.0.0.1:5000/students/1:DELETE

```
{  
  "message": "Student deleted successfully"  
}
```